Lab 00

Week 7

6733172621 Patthadon Phengpinij
Collaborators. Claude (for LaTeX styling and grammar checking)

1 Building Quantum Circuits for Real Hardware

The whole lab is done on local jupyter notebook. All codes and experiments are performed in the notebook provided in the appendix.

The answers and code solutions are provided
in the appendix located on the next page…

APPENDIX 1

Lab 00 | Code Solution

lab-00

September 20, 2026

1 Lab 0: Building Quantum Circuits for Real Hardware

1.1 Installation

Run this cell first to check your Python version.

[5]:
```
import sys

recommended = (3, 10)
current = sys.version_info[:2]

if current < recommended:
    print(f"You are using Python {current[0]}.{current[1]}. Python {recommended[0]}.
  ↪{recommended[1]} or later is recommended.")
else:
    print(f"Python {current[0]}.{current[1]} — looks good!")
```

```
Python 3.10 — looks good!
```

Next, run the cell below to install and upgrade the required packages. If the upgrade process causes dependency conflicts with packages already installed in your local environment, **create a new Python environment** (Python 3.12 recommended) and continue the challenge there.

For guidance on setting up a clean environment and installing Qiskit, refer to the IBM Quantum documentation: https://quantum.cloud.ibm.com/docs/guides/install-qiskit

[6]:
```
# Install required packages.
# Uncomment the lines below the first time you run this notebook to install dependencies,
# then re-comment them on subsequent runs.

%pip install --upgrade -q 'qiskit[visualization]>=2.5.0' qiskit-aer qiskit-ibm-runtime matplotlib
%pip install --upgrade -q qc-grader

%pip install --upgrade -q matplotlib
```

```
Note: you may need to restart the kernel to use updated packages.
Note: you may need to restart the kernel to use updated packages.
Note: you may need to restart the kernel to use updated packages.
```

Once you have installed and upgraded the libraries, please restart the Python kernel and rerun the notebook starting from the Import Libraries cell below.

[7]:
```
import os
import platform
import shutil
import subprocess
import sys

import qc_grader
import qiskit

from qc_grader.challenges.qgss_2026 import check_progress
```

```
from qc_grader.challenges.qgss_2026.lab0 import (
    grade_lab0_ex1,
    grade_lab0_ex2,
    grade_lab0_ex3,
    grade_lab0_ex4,
)


from qiskit.circuit import QuantumCircuit
from qiskit.quantum_info import SparsePauliOp, Statevector
from qiskit.transpiler.preset_passmanagers import generate_preset_pass_manager
from qiskit.visualization import array_to_latex, plot_histogram, plot_state_qsphere
from qiskit_aer import AerSimulator
from qiskit_ibm_runtime import Batch, QiskitRuntimeService
from qiskit_ibm_runtime import EstimatorV2 as Estimator
from qiskit_ibm_runtime import SamplerV2 as Sampler

import matplotlib.pyplot as plt

print(f"Qiskit version: {qiskit.__version__}")
print(f"Grader version: {qc_grader.__version__}")
```

```
Qiskit version: 2.5.2
Grader version: 2026.8.28
```

---

2 Chapter 1 — Your first quantum program with Qiskit patterns

2.1 1.1 IBM Quantum Account

First things first! To participate in the main event, you'll need an active IBM Quantum account.

If you've already saved your account credentials with `QiskitRuntimeService`, you can just run the cell below and your backend list will appear. If you're a new user, or it's been a while since you last set things up, follow these steps to save your account:

1. Go to quantum.cloud.ibm.com and sign in (or create an account).
2. Copy your API key by clicking `Create +` or `View all` on the dashboard.
3. Copy your instance CRN from the Instances tab.
4. Replace the placeholders in the cell below with your API key and Instance CRN, then run it.

Once that's done, run the grader cell to confirm your credentials are set up correctly.

[8]:
```
# Paste your IBM Cloud credentials here. You only need to do this once per environment.
# After it works, you can comment these lines out so the credentials do not stay in the file.

token = "hB6NZwsjbCCDKkjC7tg9_nX8vi9tDjMqLYbJwLg6y-pW"
instance = "crn:v1:bluemix:public:quantum-computing:us-east:a/d259ca16c69d4ec38f7331de895f2658:
  ↪0a8f47c6-38d5-4b1b-8be7-ed7085819919::"

QiskitRuntimeService.save_account(
   token=token,
   instance=instance,
   overwrite=True,
   set_as_default=True,
)

# Verify
service = QiskitRuntimeService()
backends = service.backends()
print(f"Account OK. {len(backends)} backend(s) available:")
for b in backends[:5]:
    print(f"  {b.name} ({b.num_qubits} qubits)")
```

```
Account OK. 3 backend(s) available:
  ibm_fez (156 qubits)
  ibm_marrakesh (156 qubits)
  ibm_kingston (156 qubits)
```

2.2 1.2 Quantum Computing with Gates

IBM quantum computers are gate-based. We build quantum algorithms by composing quantum gates, much like classical computers use AND, OR, and NOT gates to process bits. A key difference is that quantum gates work on qubits, the quantum counterpart of classical bits. Classical bits are strictly 0 or 1, but qubits can exhibit three quantum mechanical phenomena: superposition (a qubit exists in a linear combination of 0 and 1 until measured), entanglement (two or more qubits share a joint quantum state that cannot be described by the individual qubits independently), and interference (quantum amplitudes can add constructively or destructively, allowing algorithms to suppress wrong answers and reinforce correct ones). These three phenomena are what make quantum computing different from classical computing.

2.2.1 Superposition and the Hadamard Gate

The Hadamard gate (H) takes a qubit in a definite state ($|0\rangle$ or $|1\rangle$) and puts it into an equal superposition of both:

$$H = \frac{1}{\sqrt{2}}\begin{pmatrix}1 & 1\\ 1 & -1\end{pmatrix}, \qquad H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle)$$

When you measure this state, you get 0 or 1 with equal probability (50% each). The $\frac{1}{\sqrt{2}}$ coefficients are probability amplitudes, and the measurement probability of each outcome is the square of its amplitude.

Complete the cell below to build your first 1-qubit circuit with a Hadamard gate.

To learn more about superposition, entanglement, and interference, see the Basics of quantum information course.

Exercise 2: Build your first quantum circuit

Below is an empty 1-qubit circuit. Add a Hadamard gate on qubit 0 to put it into superposition. This is what gives you the characteristic 50/50 split when we run the Sampler in Section 1.6.

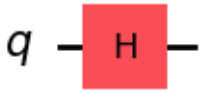
Hint: try our official QuickStart.

[9]:
```
qc = QuantumCircuit(1)

# TODO: Add a Hadamard gate on qubit 0
qc.h(0)

qc.draw("mpl")
```

[9]:



2.3 1.3 What is a Qiskit pattern?

We now have a quantum circuit that prepares the $H|0\rangle$ superposition state. In the next sections, we will run two experiments on this circuit: one to collect measurement counts, and one to compute the expectation value of the Z observable. Both experiments follow the same four-step workflow called a Qiskit pattern.

Step	What you do	In this lab
1. Map	Map the problem to quantum circuits and operators.	We build the H circuit and define the Z observable.
2. Optimize	Optimize the circuit for target hardware.	We use `generate_preset_pass_manager` to do this.
3. Execute	Execute on target hardware using primitives.	We run the circuit with both Sampler and Estimator.
4. Post-process	Post-process results.	We plot the measurement histogram and verify that Z 0.

For the Execute step, Qiskit provides two primitives:

- SamplerV2 samples the output register from quantum circuit execution. We use it to collect measurement counts and verify that 0 and 1 appear with roughly equal frequency.
- EstimatorV2 computes expectation values of observables with respect to states prepared by quantum circuits. We use it to evaluate $\langle Z \rangle$, where $|0\rangle$ has eigenvalue $+1$ and $|1\rangle$ has eigenvalue -1, giving $\langle Z \rangle = 0$ for an equal superposition.

The next sections go through each step.

2.4 1.4 Step 1 — Map

In the Map step, we define the quantum problem as a circuit and an observable.

The circuit is already built: `qc` prepares the $H|0\rangle$ state. Here we define the observable. We use the Z operator (Pauli-Z), which has eigenvalue $+1$ for $|0\rangle$ and -1 for $|1\rangle$:

$$Z = |0\rangle\langle 0| - |1\rangle\langle 1|$$

For the $H|0\rangle$ state, both outcomes are equally likely, so the expectation value is:

$$\langle Z \rangle = \frac{1}{2}(+1) + \frac{1}{2}(-1) = 0$$

This is what we will verify with Estimator in the Execute step.

Exercise 3: Define the observable

In the Map step, you choose what to measure. We want the Pauli-Z operator on qubit 0; its expectation value tells us how biased the qubit is toward $|0$ vs $|1$.

Create a single-qubit Z **observable** using SparsePauliOp with a coefficient 1.0.

Hint: you can find nice examples in our SparsePauliOp documentation.

[ ]:
```
# TODO: Define the observable as the single-qubit Z operator
observable = SparsePauliOp("Z", coeffs=[1.0])

print(f"Observable: {observable}")
```

```
Observable: SparsePauliOp(['Z'],
              coeffs=[1.+0.j])
```

2.5 1.5 Step 2 — Transpile

Quantum hardware has constraints. Real quantum hardware has a native gate set and a qubit connectivity that are determined by the underlying physical implementation. The `Transpile` step rewrites your abstract circuit into an equivalent circuit that the target backend can actually execute, using its supported gate set and connectivity.

We use `generate_preset_pass_manager` to handle this. In the cells below, we target an ideal simulator, which accepts nearly any gate, so the transpiled circuit will look close to the original. In the optional Section 1.8, we run on real hardware where the transpiler does more work: decomposing gates into the backend's native gate set and optimizing the circuit layout according to the connectivity constraints.

Transpilation applies to both the circuit and the observable. When the transpiler assigns virtual qubits to physical qubits, the observable must reflect that same mapping. We do this with `apply_layout`. Since our circuit has only one qubit, the layout is trivial here, but the same pattern is required for multi-qubit circuits on real hardware.

[11]:
```
backend = AerSimulator()
pm = generate_preset_pass_manager(optimization_level=1, backend=backend)

# Sampler requires measurements in the circuit
qc_meas = qc.copy()
qc_meas.measure_all()
isa_qc_meas = pm.run(qc_meas)

# Estimator takes the circuit without measurements and evaluates the observable separately
isa_qc = pm.run(qc)

# Apply the same layout to the observable so qubit indices match the transpiled circuit
isa_observable = observable.apply_layout(isa_qc.layout)

print("Transpiled circuit and observable ready.")
```

```
Transpiled circuit and observable ready.
```

2.6 1.6 Step 3 — Execute

Now we run the transpiled circuit using both primitives. Each answers a different question about the same quantum state.

2.6.1 Sampler: "What counts do I get?"

The Sampler runs your circuit many times (shots) and counts how often each outcome occurs: how many times do I measure 0, and how many times do I measure 1?

[12]:
```
sampler = Sampler(mode=backend)
sampler_result = sampler.run([isa_qc_meas], shots=1000).result()
counts = sampler_result[0].data.meas.get_counts()

print("Sampler counts:", counts)
```

```
Sampler counts: {'1': 505, '0': 495}
```

2.6.2 Estimator: "What value do I get when I observe my quantum circuit?"

Estimator takes the circuit and the observable and returns the expectation value directly, without you having to collect and average counts manually. The circuit does not need measurement gates — Estimator handles the measurement internally.

For the $H|0\rangle$ state, we expect $\langle Z \rangle = 0$.

[13]:
```
# Run Estimator — circuit WITHOUT measurements + observable
estimator = Estimator(mode=backend)
estimator_result = estimator.run([(isa_qc, isa_observable)]).result()
exp_val = estimator_result[0].data.evs

print(f"□Z□ = {exp_val:.4f}")
```

```
□Z□ = -0.0273
```

2.7 1.7 Step 4 — Post-process

The Post-process step is where we extract meaning from the raw results. Depending on the problem, this can mean visualizing measurement distributions, computing derived quantities from expectation values, feeding results into a classical optimization loop, or applying error mitigation techniques.

In this lab, we keep it simple. We plot the Sampler counts as a histogram to see how often 0 and 1 were measured, and we plot the Estimator result as a bar chart with an error bar showing the statistical uncertainty on $\langle Z \rangle$. On the simulator we expect equal counts and $\langle Z \rangle \approx 0$. On real hardware, both plots will reflect the effect of shot and device noise.

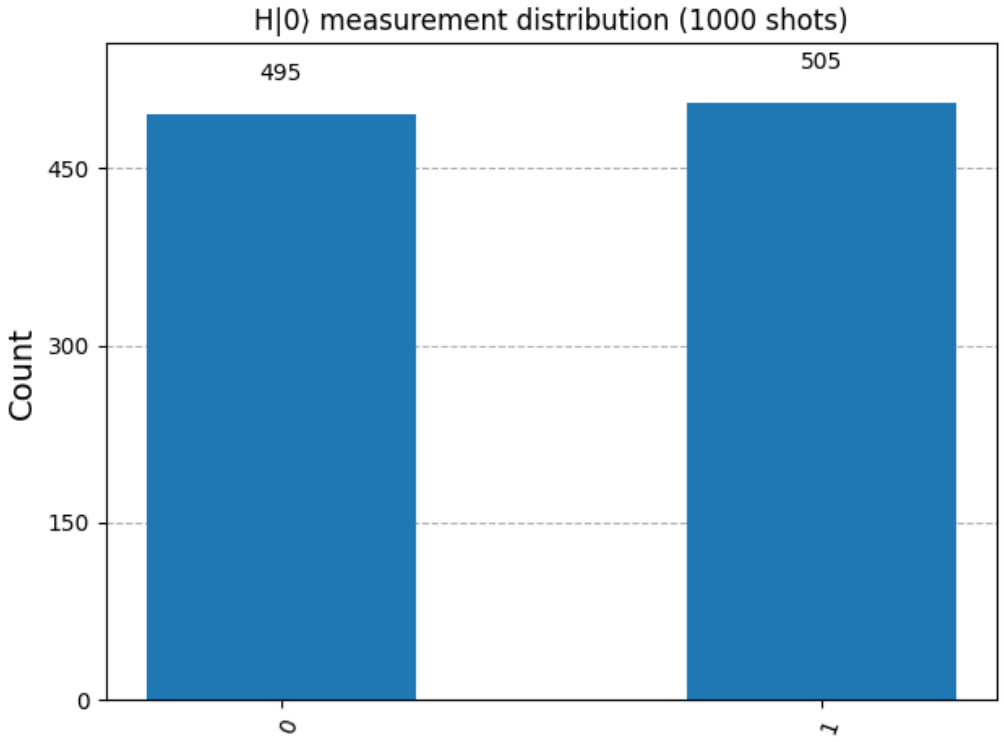
[14]:
```
# Sampler results
print(f"Sampler counts: {counts}")
display(plot_histogram(counts, title="H|0□ measurement distribution (1000 shots)"))

# Estimator results
ev = estimator_result[0].data.evs.item()
std = estimator_result[0].data.stds.item()
print(f"\nEstimator □Z□ = {ev:.4f} ± {std:.4f}")

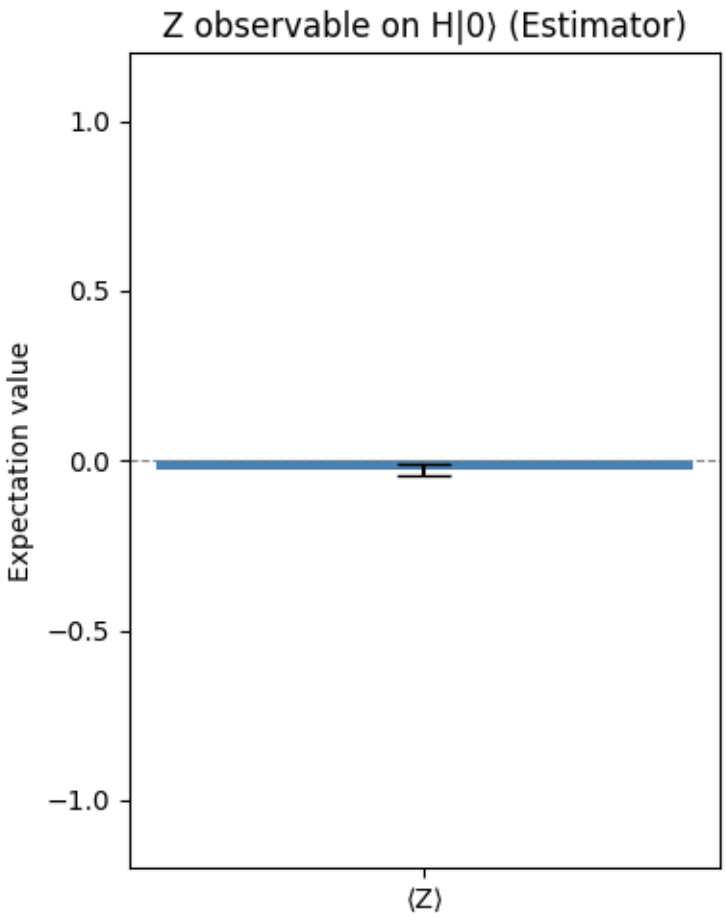
fig, ax = plt.subplots(figsize=(4, 5))
ax.bar(["□Z□"], [ev], yerr=[std], capsize=10, color="steelblue", width=0.4)
ax.axhline(0, color="gray", linewidth=0.8, linestyle="--")
ax.set_ylim(-1.2, 1.2)
ax.set_ylabel("Expectation value")
ax.set_title("Z observable on H|0□ (Estimator)")

plt.tight_layout()
plt.show()
```

```
Sampler counts: {'1': 505, '0': 495}
```




```
Estimator □Z□ = -0.0273 ± 0.0156
```



2.8 1.8 Run on Real Quantum Hardware

So far we used a simulator. Now we run the same circuit on a real IBM quantum processor. The code follows the same Qiskit Patterns workflow. The only change is the backend.

Real quantum hardware operates under physical constraints that simulators do not have. Gates run as microwave pulses on physical qubits, and each operation has a finite fidelity. As a result, the counts will be close to 50/50 but not exact, and $\langle Z \rangle$ will be close to 0 with a measurable uncertainty. Comparing these results with the simulator gives you a concrete sense of where quantum error mitigation becomes useful.

We submit both Sampler and Estimator jobs together using Batch. Batch groups multiple jobs into a single backend reservation so they run back-to-back without re-queuing. This job uses approximately 20 seconds of QPU time with `Heron` devices. Queue wait time varies depending on backend load. If the cell times out before printing the job IDs, the following cell shows how to retrieve the results by job ID.

[15]:
```
# By default, this section runs on real quantum hardware.
# To save your QPU time for the main event, uncomment the noisy simulator line below
# and replace real_backend with real_backend_sim in the rest of this section.

real_backend = service.least_busy()
# real_backend_sim = AerSimulator.from_backend(real_backend)  # noisy simulator

print(f"Running on: {real_backend.name} ({real_backend.num_qubits} qubits)")

# Step 2: Transpile for the target backend
pm_real = generate_preset_pass_manager(optimization_level=1, backend=real_backend)
```

```
qc_meas = qc.copy()
qc_meas.measure_all()
isa_qc_meas = pm_real.run(qc_meas)

isa_qc_real = pm_real.run(qc)
isa_obs = observable.apply_layout(isa_qc_real.layout)

# Step 3: Execute — both primitives run in a single Batch to avoid re-queuing
with Batch(backend=real_backend):
    sampler = Sampler()
    sampler.options.environment.job_tags = ["qgss26"]
    sampler_job = sampler.run([isa_qc_meas], shots=1000)

    estimator = Estimator()
    estimator.options.environment.job_tags = ["qgss26"]
    estimator_job = estimator.run([(isa_qc_real, isa_obs)])

sampler_job_id = sampler_job.job_id()
estimator_job_id = estimator_job.job_id()

print(f"Sampler job ID:   {sampler_job_id}")
print(f"Estimator job ID: {estimator_job_id}")
```

```
Running on: ibm_marrakesh (156 qubits)
Sampler job ID:   danphhg2fm4c73f4hpi0
Estimator job ID: danphi02fm4c73f4hpig
```

[17]:
```
# (Optional) If the cell above timed out before printing the results, copy the
# Sampler/Estimator job IDs from its output into the strings below and run this cell.
# Leave commented otherwise.

# sampler_job_id = "<paste sampler job id>"
# estimator_job_id = "<paste estimator job id>"
# sampler_job = service.job(sampler_job_id)
# estimator_job = service.job(estimator_job_id)
```

[16]:
```
# Step 4: Results
hw_counts = sampler_job.result()[0].data.meas.get_counts()
hw_estimator_result = estimator_job.result()
hw_exp_val = hw_estimator_result[0].data.evs

print(f"Hardware Sampler counts: {hw_counts}")
print(f"Hardware Estimator □Z□:  {hw_exp_val:.4f}")
```

```
Hardware Sampler counts: {'0': 487, '1': 513}
Hardware Estimator □Z□:  0.0115
```

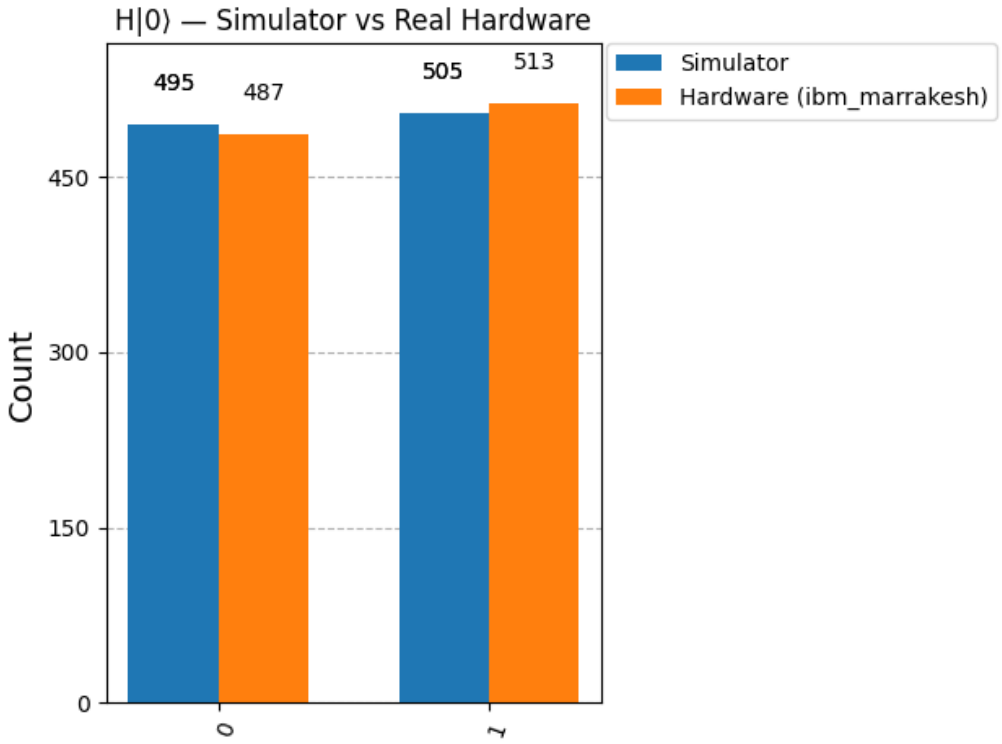
Compare the results side by side. The simulator gives a 50/50 split. The hardware results show the effect of noise: small deviations from it.

[18]:
```
display(plot_histogram(
    [counts, hw_counts],
    legend=["Simulator", f"Hardware ({real_backend.name})"],
    title="H|0□ — Simulator vs Real Hardware",
))

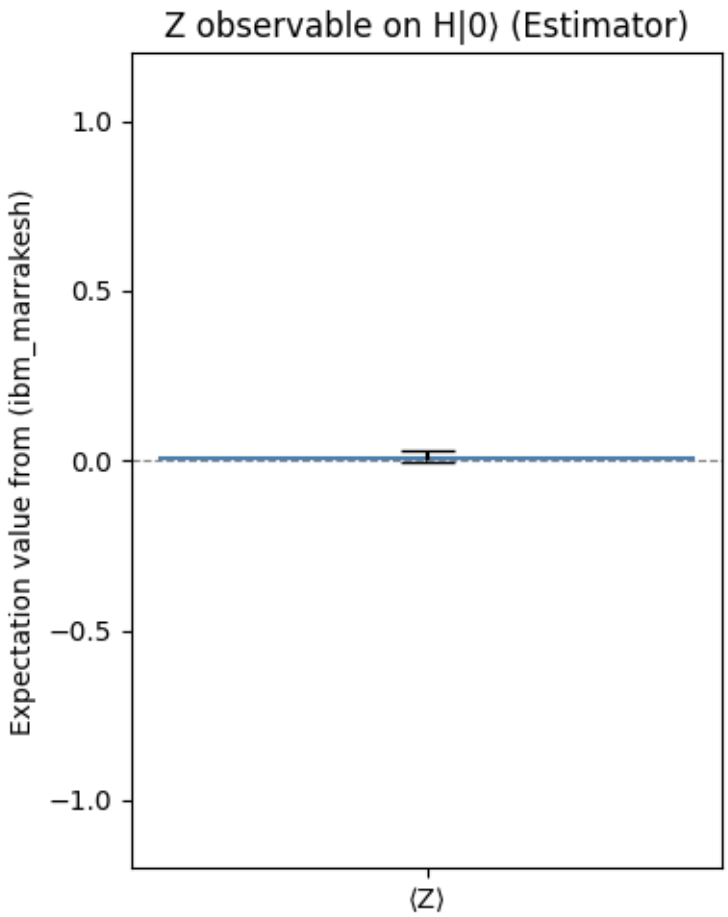

# Estimator results
ev = hw_estimator_result[0].data.evs.item()
std = hw_estimator_result[0].data.stds.item()
print(f"\nEstimator □Z□ = {ev:.4f} ± {std:.4f}")
```

```
fig, ax = plt.subplots(figsize=(4, 5))
ax.bar(["□Z□"], [ev], yerr=[std], capsize=10, color="steelblue", width=0.4)
ax.axhline(0, color="gray", linewidth=0.8, linestyle="--")
ax.set_ylim(-1.2, 1.2)
ax.set_ylabel(f"Expectation value from ({real_backend.name})")
ax.set_title("Z observable on H|0□ (Estimator)")

plt.tight_layout()
plt.show()
```




```
Estimator □Z□ = 0.0115 ± 0.0170
```



3 Chapter 2 — Multi-qubit circuit and visualization

Great work on Chapter 1! You built a single-qubit superposition, ran it through Sampler and Estimator, and saw how the same quantum state looks from two different measurement perspectives — on both a simulator and, if you tried it, real quantum hardware.

Now let's build something more interesting. In this chapter, you will work with 6 qubits to construct a special quantum circuit step by step. At the end, we will look at the statevector it produces and visualize it using QSphere, a multi-qubit visualization tool that shows the probability and phase of each basis state. You might recognize what the final state looks like.

3.1 2.1 Build a 6-qubit entangled state

You will use three gates in this exercise:

- H: puts a qubit into an equal superposition of $|0\rangle$ and $|1\rangle$ (you already know this one).
- X: flips a qubit from $|0\rangle$ to $|1\rangle$ and vice versa. It is the quantum equivalent of a classical NOT gate.
- CX: a two-qubit gate that flips the target qubit if and only if the control qubit is $|1\rangle$. Applying CX to a superposition state creates entanglement between the two qubits.

If you want to explore these gates further and run them on real hardware, check out Build and run your first quantum program.

All qubits start in $|0\rangle$. Start with the cell below.

Exercise 4: Build a 6-qubit entangled state

You will build a 6-qubit circuit. When you visualize it on a QSphere later, you will see something familiar.

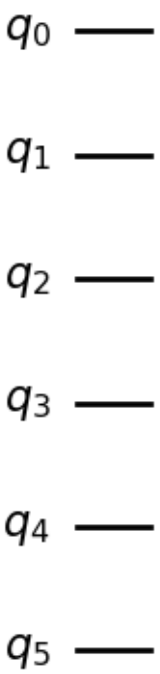
Build it in three steps using the cells below:

1. (already given) Create a 6-qubit circuit.
2. Apply H to qubit 0 to put it in superposition, then apply X to qubit 1.
3. Apply a CNOT chain with qubit 0 as control and qubits 1 through 5 as targets to entangle them all.

Hint: you can use a `for` loop to build the CNOT chain.

[19]:
```
# Step 1: Create a 6-qubit circuit
special_qc = QuantumCircuit(6)
special_qc.draw("mpl")
```

[19]:



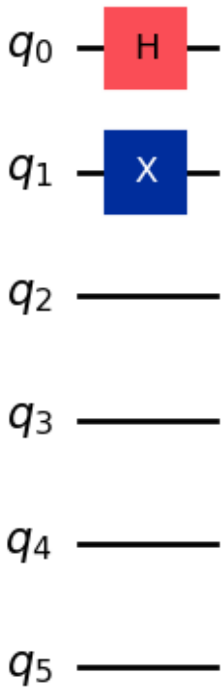
Next, apply H to qubit 0, putting it into superposition. Then apply X to qubit 1, flipping it from $|0\rangle$ to $|1\rangle$. The circuit now has two qubits prepared in specific states, ready for entanglement in the next step.

[20]:
```
# Step 2: H on qubit 0, X on qubit 1
# TODO: Apply H to qubit 0
special_qc.h(0)

# TODO: Apply X to qubit 1
special_qc.x(1)

special_qc.draw('mpl')
```

[20]:

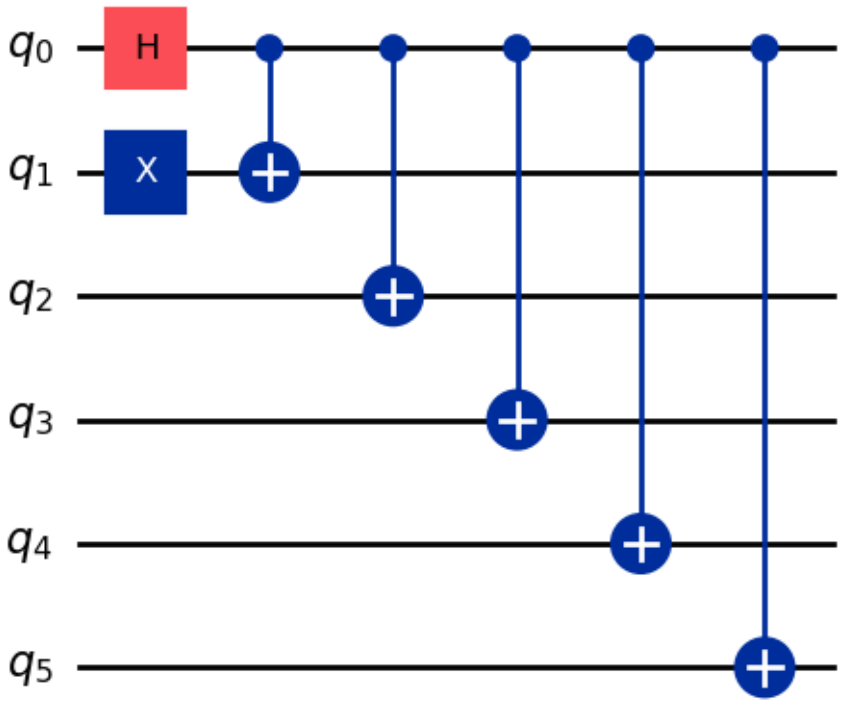


Now we create entanglement using a CNOT chain. Each CNOT gate uses qubit 0 as the control and targets qubits 1 through 5. This links all qubits together: when qubit 0 is in superposition, all other qubits become entangled with it. The circuit diagram shows five CNOT gates connecting qubit 0 to every other qubit.

[21]:
```
# Step 3: CNOT chain — entangle all qubits with qubit 0
# TODO: Loop over qubits 1 through 5 and apply CNOT with qubit 0 as control
for target in range(1, 6):
    special_qc.cx(0, target)

special_qc.draw('mpl')
```

[21]:



3.2 2.2 Statevector

The statevector is the complete mathematical description of a quantum state. For n qubits, it is a complex vector of 2^n amplitudes, one for each possible basis state. The probability of measuring a particular basis state is the square of its amplitude.

For our 6-qubit circuit, the statevector has $2^6 = 64$ entries, covering every basis state from $|000000\rangle$ to $|111111\rangle$. The `Statevector` class computes this from the circuit, and `array_to_latex` prints it in mathematical form.

Most entries will be zero. Only the basis states with non-zero amplitude contribute to the quantum state you built.

[22]:
```
sv = Statevector(special_qc)
array_to_latex(sv, max_size=64)
```

[22]:

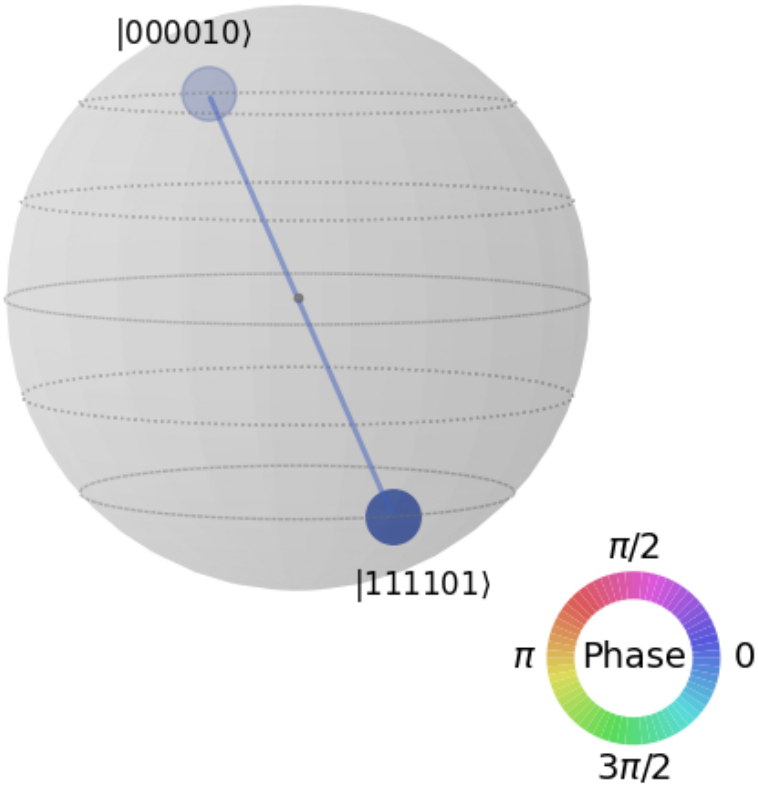
$$\left[0 \quad 0 \quad \frac{\sqrt{2}}{2} \quad 0 \quad 0 \quad 0

- Color represents the quantum phase.

For the state you built, you will see two dots of equal size. Take a look at which basis states they correspond to.

[23]:
```
# The QSphere
plot_state_qsphere(sv)
```

[23]:



3.3.1 What the QSphere shows

The QSphere has two dots, one near the top and one near the bottom. They are the two basis states in equal superposition, linked by entanglement.
